

WordForge

1. Project Overview

Provide a brief overview of your project, explaining the game's concept, mechanics, and functionality.

This project is a word-puzzle battle game inspired by [Bookworm Adventures](#), in which players form words from a grid of letter tiles to attack enemies. The player connects adjacent letters to create valid words, which deal damage based on word length and tile effects. The game features an infinite-level system in which enemy difficulty increases via mathematical scaling. Special colored tiles provide additional effects such as bonus damage, healing, or score multipliers to add strategic gameplay.

2. Project Review

Study and then present a relevant existing project, outlining the specific improvements you plan to make. Highlight the key areas where your project enhances or builds upon the original.

An existing project relevant to this game is [Bookworm Adventures](#), a word puzzle RPG where players create words from letter tiles to attack enemies and progress through predefined stages. The game also includes various passive skills and special bonuses tied to specific word types.

The proposed project improves on this concept by introducing an [infinite-level system](#) where enemy difficulty increases dynamically using mathematical calculations instead of fixed stages. Additionally, the design simplifies gameplay by [removing passive skill mechanics](#) while still keeping [special colored tiles](#) that provide strategic effects. This creates a more scalable and focused gameplay experience while maintaining the core word-puzzle combat mechanic.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

The game is a word-puzzle battle game in which players create words by connecting adjacent letters on a grid of tiles. Each valid word deals damage to an enemy based on the word length and special tile effects. The game features an infinite-level system in which enemy difficulty increases through mathematical scaling. Special colored tiles provide additional effects such as extra damage, healing, or score bonuses, adding strategic elements to the gameplay.

3.2 Object-Oriented Programming Implementation

*List and describe **at least five classes** you will implement in your game. For each class, briefly explain its role, key attributes, and methods. You can also include a UML diagram for better understanding. (Later, this UML will be required as part of the full proposal.)*

1. Game Class - Controls the overall game flow, including starting the game, updating the game state, managing levels, and coordinating data collection.

Key Attributes:

- current_level
- player
- enemy
- game_board
- data_collector

Methods:

- start_game()
- update_game_state()
- next_level()
- end_game()

2. Player Class - Represents the player character and manages player-related statistics such as health, score, and word-based attacks. It also triggers data recording when the player creates words or deals damage.

Key Attributes:

health
score
words_created
game

Methods:

attack_enemy()
receive_damage()
heal()

3. Enemy Class - Represents enemies that the player must defeat by forming words. Enemy difficulty increases with each level.

Key Attributes:

health
attack_damage
level

Methods:

attack_player()
receive_damage()
scale_difficulty()

4. Tile Class - Represents individual letter tiles on the game board, including their letter and special effects.

Key Attributes:

letter
tile_color
special_effect

Methods:

activate_effect()
display_tile()

5. Board Class - Manages the grid of letter tiles used for word formation and detects tile interactions such as clicks.

Key Attributes:

grid
board_size

Methods:

```
generate_tiles()
remove_tiles()
drop_tiles()
refill_board()
```

6. DataCollection Class - Handles recording of gameplay data using an event-based system. Each event is stored with a timestamp and saved into a CSV file for later analysis. It records raw data such as clicks, word length, damage dealt, damage received, and level instead of pre-calculated statistics.

Key Attributes:

```
data
```

Methods:

```
log_event()
save_to_csv()
```

3.3 Algorithms Involved

Mention the algorithms used in the game. If your game incorporates techniques such as pathfinding, sorting, AI behavior, rule-based logic, or event-driven mechanics, describe them here.

1. Word Validation Algorithm

The game uses a dictionary-based search algorithm to verify whether a word created by the player is valid. To improve performance, the system first filters the dictionary by selecting only words that contain the same letters as the player's input. For example, if the player forms the word "sand", the algorithm searches within a subset of the dictionary starting with the letter s only. This will reduce unnecessary comparisons.

2. Board Generation Validation Algorithm

Before new tiles are added to the board, the game checks whether at least one valid word can be formed from the current tile arrangement. If no possible word exists, the system regenerates or adjusts the tiles until a valid word combination is guaranteed. This prevents situations where the player cannot make any move.

3. Board Tile Update Algorithm

After a valid word is formed, the selected tiles are removed from the grid. A tile-drop algorithm shifts tiles downward to fill empty spaces, similar to

gravity mechanics in puzzle games. New tiles are then generated to refill the board.

4. Level Scaling Algorithm

Enemy difficulty increases dynamically using a mathematical scaling formula. Enemy health and damage increase as the level number grows, allowing the game to support an infinite level progression system.

5. Rule-Based Tile Effect Algorithm

Special colored tiles trigger specific effects when used in a word. A rule-based logic system checks the tile type and applies effects such as bonus damage, healing, or score multipliers during the damage calculation process.

4. Statistical Data (Prop Stats)

4.1 Data Features

Describe *at least five features* you will track in the game, which may include player behavior or game-related metrics (e.g., character movement, keystrokes, pixels moved, player score, time played, enemies defeated, accuracy, and completion rate). Each feature should have *at least 100 rows* of data. (For example, in football, a player’s performance can be analyzed by collecting and analyzing statistical data.)

Feature	Why is it useful?	How to get 100 values?	Variable & Class	Display
Tile Clicking (every 20s)	Detects spam vs strategic play	Count clicks every 20s across sessions	DataCollection (click)	Line Graph (Time vs Clicks)
Words Created (by length)	Shows preferred word length	Record words and group by length	DataCollection (word_length)	Table (length, count)
Damage Dealt (overall)	Shows damage distribution	Record all damage events	DataCollection (damage_dealt)	Histogram (Damage vs Frequency)
Damage Dealt (every 20s)	Shows combat consistency	Sum damage every 20s	DataCollection (damage_dealt)	Boxplot (Damage distribution)

Damage Received (every 20s)	Shows difficulty and survivability	Sum damage received every 20s	DataCollection (damage_received)	Scatter Plot (Time vs Damage)
Time to Complete Level	Shows level difficulty	Group level times (<1, 1-5, >5 min)	DataCollection (level, timestamp)	Bar Graph (Time category vs Count)

4.2 Data Recording Method

Explain how the game will store statistical data. Will it be saved in a database, CSV file, or another format?

The game will store statistical data in a CSV file using an event-based recording system. Each row represents a single gameplay event with a timestamp.

CSV columns (header):

- timestamp - time when the event occurs
- event_type - type of event (click, word_created, damage_received, level_change)
- click - 1 if a tile is clicked, otherwise 0
- word_length - length of the created word
- damage_dealt - damage dealt to the enemy
- damage_received - damage taken from the enemy
- level - current level

The system records raw event data instead of pre-calculated metrics. Statistical features such as clicks every 20 seconds, damage per interval, and word length distribution will be calculated later during data analysis by grouping and filtering the recorded data.

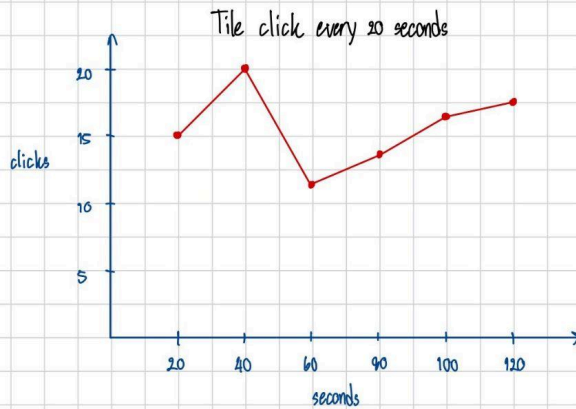
4.3 Data Analysis Report

Outline how you will analyze the recorded data. What statistical measures will you use? How will the analysis be presented (e.g., graphs, tables, charts)?

The recorded gameplay data will be analyzed using both statistical measures and data visualization techniques. The analysis will focus on identifying player behavior patterns, performance trends, and game balance across different levels.

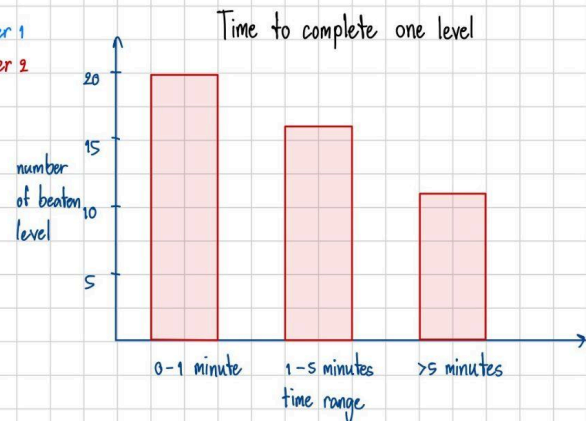
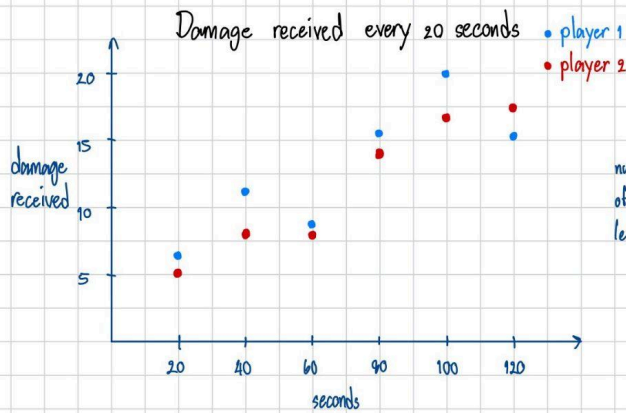
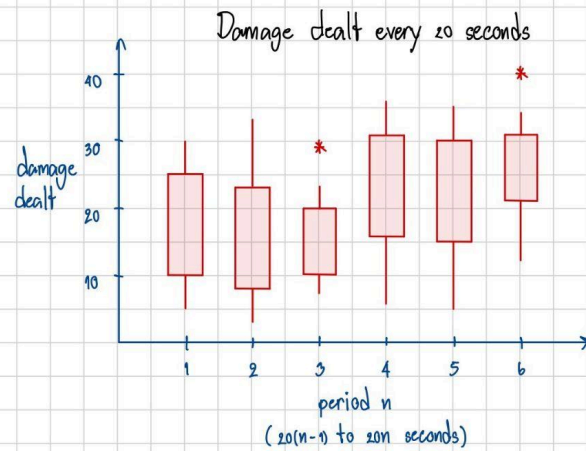
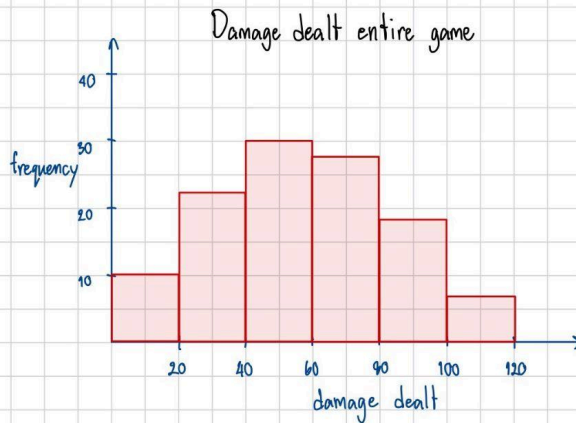
Feature	What the Graph Shows / Why	Visualization Method	X-axis	Y-axis
Tile Clicking (every 20s)	Shows player activity trend and detects spam vs strategic play	Line Graph	Time (20-sec intervals)	Number of clicks
Words Created (by length)	Shows the distribution of word lengths used by the player	Table	Word length	Number of words
Damage Dealt (overall)	Shows the distribution of damage values	Histogram	Damage dealt	Frequency
Damage Dealt (every 20s)	Shows variation and spread of damage over time	Boxplot	Time intervals	Damage dealt
Damage Received (every 20s)	Shows the relationship between time and damage taken	Scatter Plot	Time (20-sec intervals)	Damage received
Time to Complete Level	Compares the number of levels in each time category	Bar Graph	Time category (<1, 1-5, >5 min)	Number of levels

Hand-drawn expected graphs



Word created for each length

Length	Word list	Number
3	pie, fox	2
4	same, type, loan	3
5	lotus, grade, fails, loans	4
6	travel, silver	2
7	journey, freedom	2
8	freedoms, mountain, complete	3
10	background, adventure	2



5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement basic system (No pygame GUI)
3	10	Create a simple GUI for level 1 gameplay
4	11	Implement special tiles and level scaling
5	12	Implement stats recording
6	13	Let friends test the game and provide feedback.
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 13 March 2025

Editor: Wassawin Swangjaeng

Version: 2.0

Date: 28 March 2025

Editor: Wassawin Swangjaeng

Date	Name	Feedback
21/03	Amornrit	Great detail on the algorithm part. Justify your proposed data features. Provide more details on the Data Analysis Report, how your proposed data features will be analyzed and visualized.